

实验五：密文卷积的实现及结果正确性验证

一、实验目标

本实验选择开源全同态加密库 TenSEAL（基于 Microsoft SEAL），在 CKKS 近似同态加密方案下实现单输入单输出的密文卷积运算，具体目标为：

1. 使用 4×4 输入矩阵与 3×3 卷积核（步长 1、无填充），完成密文域上的卷积计算；
2. 采用 im2col 思想将滑动窗口展开为向量，通过密文-明文点积实现卷积；
3. 解密结果并与明文标准卷积结果对比，验证正确性（允许 CKKS 近似误差在 $1e-3$ 以内）；
4. 掌握 CKKS 上下文参数设置、向量加密、同态乘法与累加的基本流程。

二、实验原理

2.1 CKKS 全同态加密方案

CKKS（Cheon-Kim-Kim-Song）是一种支持近似实数/复数运算的全同态加密方案，特别适合机器学习中的浮点计算。它将向量编码到多项式环中，支持同态加法、同态乘法与旋转操作。由于使用近似算术，解密结果存在可控噪声误差。

2.2 卷积的 im2col 实现

传统卷积对每个输出位置取输入局部窗口与卷积核做点积。im2col 将所有滑动窗口展平为行向量，形成矩阵，卷积核展平为列向量，通过一次矩阵-向量乘法即可得到所有输出。在同态加密中，可对每个窗口向量单独加密并与明文核做点积，避免复杂的密文-密文运算。

2.3 本实验参数

- 输入： 4×4 矩阵，元素为 1.0 至 16.0 的连续浮点数；
- 卷积核： 3×3 ，形如 $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ （垂直边缘检测类核）；
- 输出尺寸： 2×2 （步长 1、无填充）；
- 加密方案：

CKKS, poly_modulus_degree=8192, coeff_mod_bit_sizes=[60, 40, 40, 60], global_scale= 2^{40} 。

三、实验环境与设置

3.1 软件环境

- 编程语言：Python 3.12.6；
- 同态加密库：TenSEAL（封装 Microsoft SEAL 的 Python 接口）；
- 数值计算：NumPy；
- 运行环境：支持 SEAL 编译的 Windows。

3.2 上下文创建

使用 `ts.context` 创建 CKKS 上下文，生成 Galois 密钥以支持可能的旋转操作，设置全局缩放因子保证乘法后精度。

四、实验步骤与实验结果

4.1 明文基准计算

对每个 2×2 输出位置，计算对应 3×3 窗口与核的逐元素乘积之和，得到标准结果：

期望输出矩阵 (2×2):

```
[[ -6.0, -6.0 ],  
 [ -6.0, -6.0 ]]
```

(因为每个窗口左侧三列之和减去右侧三列之和均为-6)。

4.2 密文卷积流程

1. 调用 `im2col_2d` 将 4×4 输入展开为 4 个 9 维向量 (对应 4 个滑动窗口);
2. 对每个窗口向量使用 `ts.ckks_vector` 加密;
3. 将 3×3 核展平为 9 维明文列表;
4. 对每个加密窗口调用 `.dot(kernel_flat)` 完成同态点积 (内部为同态乘法+累加);
5. 解密 4 个标量结果并 `reshape` 为 2×2 矩阵。

4.3 实验结果

```
= RESTART: C:\Users\admin\Desktop\创新\  
=== 明文数据 ===  
输入矩阵 (4x4):  
[[ 1.  2.  3.  4.]  
 [ 5.  6.  7.  8.]  
 [ 9. 10. 11. 12.]  
 [13. 14. 15. 16.]]  
卷积核 (3x3):  
[[ 1.  0. -1.]  
 [ 1.  0. -1.]  
 [ 1.  0. -1.]]  
明文标准卷积计算结果 (2x2):  
[[-6. -6.]  
 [-6. -6.]]  
  
=== FHE 密文解密结果 ===  
[[-6.00000173 -6.00000171]  
 [-6.00000167 -6.0000017 ]]  
  
最大绝对误差: 0.00000173  
结论: 密文卷积计算成功, 验证正确!
```

明文标准卷积结果:

```
[[ -6. -6.]  
 [-6. -6.]]
```

FHE 密文解密结果 (近似值):

```
[[ -6.0000... -6.0000...]  
 [-6.0000... -6.0000...]]
```

最大绝对误差通常远小于 $1e-3$ (CKKS 在该参数下精度充足)。

结论：密文卷积计算成功，验证正确（`np.allclose` 通过，`atol=1e-3`）。

4.4 关键正确性保障

- 跳过透明密文问题：本实现使用密文-明文点积，核中含 0 并不直接导致 `transparent ciphertext`；
- 缩放因子与模数链匹配，保证一次乘法后仍可正确解密；
- 使用 Galois 密钥生成，为后续可能扩展旋转操作预留能力。

五、实验讨论

5.1 方案选择说明

选择 TenSEAL 的原因：API 简洁、对向量运算友好、底层基于成熟的 SEAL 库，适合教学与原型验证。相比直接使用 SEAL C++ API，Python 接口大幅降低了编码复杂度。

5.2 im2col 与打包旋转的对比

本实验采用“逐窗口加密+点积”方式，简单直观，旋转次数为 0。作业 6 将探索“单密文打包+旋转+累加”策略，可进一步减少密文数量并分析旋转次数下界。

5.3 误差来源与控制

CKKS 的近似误差主要来自编码缩放与模数切换。本参数（`scale=240`，模数链足够深）下，单次点积误差极小，完全满足 `atol=1e-3` 的验证要求。

5.4 实验心得

通过本实验掌握了全同态加密在简单 CNN 层中的应用方式，理解了密文计算与明文计算在精度、性能上的差异。im2col 思想在同态场景下依然有效，为后续更复杂的同态神经网络奠定基础。